Proposal

Pengembangan Aplikasi Student Task Manager dengan Memanfaatkan Generative AI sebagai Asisten Pemrograman

Pengembangan Aplikasi Berbasis Web

Disusun Oleh:

Toman Sihombing (11S24034)

Lecturer:

Arie Satia Dharma, S.T, M.Kom.

PROGRAM STUDI INFORMATIKA

FAKULTAS INFORMATIKA DAN TEKNIK ELEKTRO

INSTITUT TEKNOLOGI DEL

2026

1. PENDAHULUAN

1.1 Latar Belakang

Perkembangan Generative Artificial Intelligence (AI) telah mengubah cara pengembang perangkat lunak merancang, menulis, menguji, dan memperbaiki kode program. Alat AI generatif dapat menjelaskan konsep, menghasilkan potongan kode, menyarankan struktur aplikasi, serta membantu menelusuri kesalahan. Namun, keluaran AI tidak selalu benar, lengkap, atau aman. Pengembang tetap bertanggung jawab memberikan instruksi yang jelas, mengevaluasi hasil, dan memverifikasinya melalui pengujian.

Di sisi lain, mahasiswa sering menerima banyak tugas dari beberapa mata kuliah dengan tenggat yang berbeda-beda. Pencatatan yang tersebar, misalnya pada catatan ponsel, grup percakapan, dan buku, membuat tugas mudah terlewat. Aplikasi manajemen tugas yang sederhana dapat membantu mahasiswa melihat seluruh tugas dalam satu tempat. Proposal ini mengusulkan pengembangan aplikasi tersebut sekaligus sebagai sarana latihan menggunakan AI generatif secara kritis dalam pemrograman web.

1.2 Alasan Pemilihan Aplikasi

Aplikasi Student Task Manager dipilih karena relevan dengan mata kuliah PABWE dan dapat mencakup konsep inti pengembangan web: struktur halaman (HTML), tampilan responsif (CSS), logika dan interaksi (JavaScript), manipulasi DOM, penanganan formulir dan validasi, serta penyimpanan data di sisi klien dengan localStorage. Ruang lingkupnya cukup kecil untuk diselesaikan dalam satu minggu, tetapi cukup kaya untuk mendemonstrasikan proses meminta kode kepada AI, menemukan kesalahan, dan memperbaikinya. Aplikasi ini juga dapat dijalankan secara lokal tanpa server maupun basis data.

1.3 Rumusan Masalah

1. Bagaimana merancang dan membangun aplikasi web sederhana yang dapat mencatat, mengelola, dan memantau tugas perkuliahan?
2. Bagaimana memanfaatkan AI generatif sebagai asisten pemrograman secara efektif melalui instruksi (prompt) yang terarah?
3. Bagaimana mengevaluasi kode hasil AI, mengidentifikasi kesalahan, dan memperbaikinya secara terukur?

1.4 Tujuan Pengembangan

- Menghasilkan aplikasi Student Task Manager yang berfungsi penuh dan dapat dijalankan di browser.

- Mendokumentasikan instruksi kepada AI, evaluasi keluarannya, kesalahan yang ditemukan, dan perbaikan yang dilakukan.
- Menyusun pengujian yang dapat diverifikasi dan menyimpan seluruh hasil kerja pada repository GitHub.

1.5 Manfaat

Bagi pengguna, aplikasi membantu memantau tenggat dan progres tugas. Bagi penulis, proyek ini melatih keterampilan pengembangan web dan kemampuan berpikir kritis terhadap keluaran AI. Bagi pembaca, dokumentasi proses menjadi contoh praktik penggunaan AI yang bertanggung jawab.

1.6 Batasan Proyek

- Aplikasi bersifat sisi klien (client-side); tidak memakai server, basis data, atau autentikasi pengguna.
- Data tersimpan di localStorage pada browser dan perangkat yang sama; tidak ada sinkronisasi antarperangkat.
- Tidak tersedia notifikasi pengingat, kategori, atau prioritas tugas.
- Pengujian otomatis hanya mencakup logika data; tampilan (DOM) diuji secara manual.
- Penanda “Terlambat” memakai tanggal lokal perangkat (asumsi penulis; tidak diatur dalam instruksi tugas).

2. DESKRIPSI APLIKASI

2.1 Target Pengguna

Target pengguna adalah mahasiswa yang mengelola beberapa tugas perkuliahan secara bersamaan dan menggunakan browser modern pada komputer maupun ponsel.

2.2 Kebutuhan Pengguna

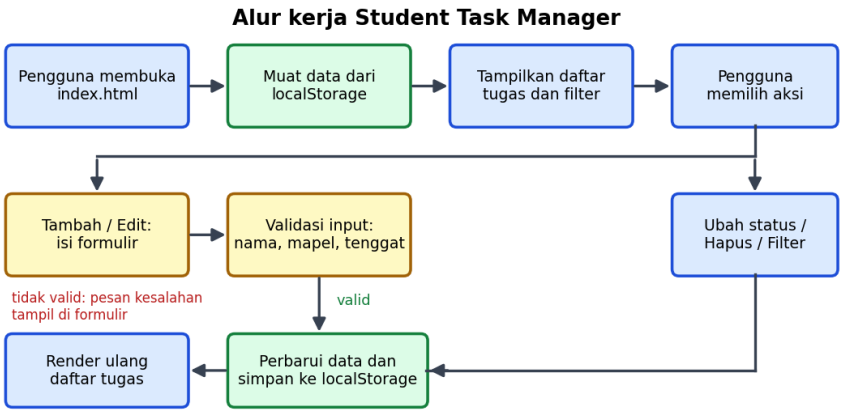
- Mencatat tugas dengan cepat beserta mata kuliah dan tenggatnya.
- Melihat daftar tugas yang terurut berdasarkan tenggat terdekat.
- Mengubah atau menghapus tugas yang salah atau sudah tidak relevan.
- Menandai tugas yang telah selesai dan menyaring daftar berdasarkan status.
- Memperoleh pesan yang jelas bila input tidak valid, dan data tidak hilang ketika halaman dimuat ulang.

2.3 Fitur Utama

No.	Fitur	Deskripsi
1	Tambah tugas	Formulir berisi nama tugas, mata kuliah, dan tenggat waktu.
2	Daftar tugas	Menampilkan nama, mata kuliah, tenggat, dan status; diurutkan menurut tenggat.
3	Edit dan hapus	Edit memuat data ke formulir; hapus meminta konfirmasi.
4	Status selesai	Kotak centang menandai selesai/belum selesai; nama tugas selesai dicoret.
5	Validasi input	Semua kolom wajib; spasi saja ditolak; batas panjang; tanggal harus valid.
6	Filter status	Semua, Belum selesai, atau Selesai.
7	Penyimpanan	Data disimpan otomatis di localStorage dan dimuat saat halaman dibuka.
8	Responsif	Tata letak menyesuaikan lebar layar desktop dan seluler.

2.4 Alur Penggunaan

Pengguna membuka index.html, aplikasi memuat data dari localStorage dan menampilkan daftar tugas. Pengguna kemudian memilih aksi: menambah atau mengedit tugas melalui formulir (dengan validasi sebelum data disimpan), atau mengubah status, menghapus, dan menyaring tugas. Setiap perubahan data disimpan ke localStorage, kemudian daftar digambar ulang. Alur ini diperlihatkan pada Gambar 1.



Gambar 1. Diagram Alur Kerja Student Task Manager

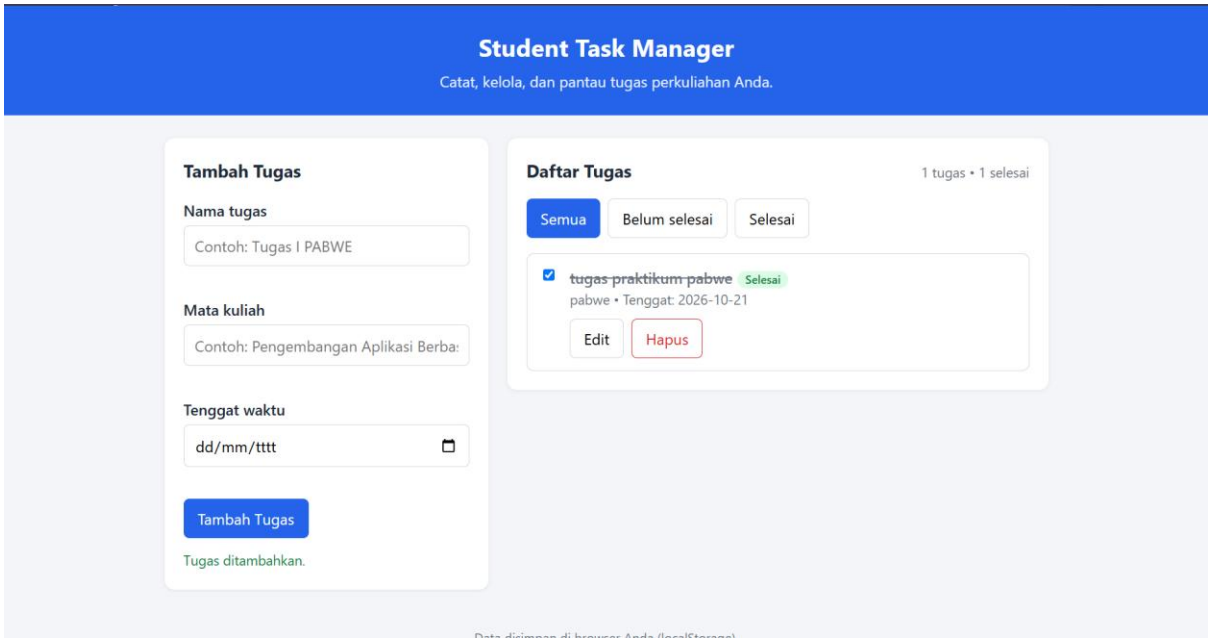
Sumber: Dibuat penulis untuk proposal ini (matplotlib; berkas docs/diagrams/alur-kerja-aplikasi.png)

Keterangan: Menunjukkan urutan pemuatan data, pemilihan aksi, validasi, penyimpanan, dan render ulang sehingga pembaca memahami alur logika yang diimplementasikan pada js/script.js.

2.5 Gambaran Hasil yang Diharapkan

Hasil akhir berupa satu halaman web dengan dua bagian: formulir tambah/edit tugas dan daftar tugas dengan tombol filter. Pada layar lebar, kedua bagian ditampilkan berdampingan;

pada layar sempit, ditampilkan bertumpuk. Gambar 2 disediakan sebagai tempat tangkapan layar aplikasi yang sebenarnya.



Gambar 2. Tampilan Aplikasi Student Task Manager

Keterangan: Menampilkan antarmuka aplikasi dengan beberapa tugas contoh. Gambar ini masih berupa placeholder dan harus diganti dengan screenshot asli setelah aplikasi dijalankan.

3. TEKNOLOGI YANG DIGUNAKAN

Teknologi	Fungsi dalam proyek
HTML5	Membangun struktur halaman: formulir, tombol filter, dan wadah daftar tugas, dengan atribut aksesibilitas seperti label dan aria-live.
CSS3	Mengatur tampilan, warna, jarak, serta tata letak responsif menggunakan CSS Grid, Flexbox, dan media query.
JavaScript	Mengelola logika: validasi, operasi tambah/edit/hapus/ubah status, penyaringan, pengurutan, dan manipulasi DOM.
localStorage	Menyimpan data tugas dalam format JSON pada browser agar tidak hilang saat halaman dimuat ulang.
Browser	Lingkungan eksekusi aplikasi; DevTools dipakai untuk menguji, memeriksa Local Storage, dan mensimulasikan layar seluler.
Node.js	Menjalankan pengujian otomatis logika data (modul bawaan assert), tanpa dependensi tambahan.
AI generatif (Claude)	Asisten untuk merancang struktur, menyusun kode awal, meninjau, dan menganalisis kesalahan. Penulis dapat mengganti dengan alat lain yang diizinkan (misalnya ChatGPT, Gemini, atau GitHub Copilot).
Git dan GitHub	Kontrol versi dan penyimpanan hasil proyek yang dapat diakses dosen dan asisten.

4. METODOLOGI PENGEMBANGAN

Pengembangan dilakukan secara iteratif dalam enam tahap seperti pada Gambar 3. Iterasi terjadi pada tahap evaluasi hingga perbaikan: bila pengujian atau tinjauan menemukan masalah, penulis menyusun prompt lanjutan, menerapkan perbaikan, dan menguji ulang.



Gambar 3. Tahapan Pengembangan dengan Bantuan AI

Sumber: Dibuat penulis untuk proposal ini (berkas docs/diagrams/tahapan-pengembangan-ai.png)

Keterangan: Memperlihatkan siklus kerja dari perencanaan hingga dokumentasi serta panah iterasi yang menekankan bahwa keluaran AI harus dievaluasi dan diperbaiki, bukan langsung diterima.

Tahap 1 – Perencanaan. Menetapkan fitur minimal, struktur folder, dan kriteria keberhasilan berdasarkan instruksi tugas, lalu menyusun prompt awal.

Tahap 2 – Pembuatan kode dengan bantuan AI. Meminta AI menghasilkan kerangka HTML, CSS, dan JavaScript berdasarkan spesifikasi fitur.

Tahap 3 – Evaluasi hasil AI. Membaca kode baris demi baris, memeriksa kesesuaian dengan spesifikasi, keamanan (misalnya penanganan input pengguna), dan penanganan kasus tepi.

Tahap 4 – Pengujian. Menjalankan pengujian otomatis (node tests/test.js) dan pengujian manual di browser sesuai skenario pada Bagian 7.

Tahap 5 – Perbaikan. Mereproduksi kesalahan, menentukan penyebab, memperbaiki kode, dan menguji ulang.

Tahap 6 – Dokumentasi. Menulis README, log penggunaan AI, rencana pengujian, serta mengunggah seluruh berkas ke GitHub dan memberikan akses kepada dosen dan asisten.

5. PEMANFAATAN GENERATIVE AI

Bagian ini memuat enam contoh prompt yang dapat diuji pada alat AI pilihan. Prompt disusun dengan konteks, batasan, dan format keluaran yang jelas. Bagian “keputusan mahasiswa” menunjukkan bahwa penentuan akhir tetap berada pada penulis.

5.1 Merancang Struktur Aplikasi

Aspek	Uraian
Tujuan instruksi	Merancang Struktur Aplikasi
Contoh prompt	Rancang struktur aplikasi web “Student Task Manager” memakai HTML, CSS, dan JavaScript murni tanpa library. Fitur: tambah, edit, hapus, tandai selesai, filter status, validasi, dan localStorage. Berikan struktur folder, daftar fungsi utama beserta tanggung jawabnya, dan bentuk objek data. Jelaskan alasan pemisahan logika dan tampilan.
Hasil yang diharapkan	Rancangan folder, daftar fungsi, dan skema objek tugas.
Cara evaluasi	Memeriksa apakah seluruh fitur minimal tercakup, apakah ada dependensi yang tidak perlu, dan apakah rancangan dapat diuji.
Keputusan mahasiswa	Menerima rancangan dasar; menambahkan pemisahan fungsi murni agar dapat diuji di Node.

5.2 Membuat Kode Awal

Aspek	Uraian
Tujuan instruksi	Membuat Kode Awal
Contoh prompt	Berdasarkan rancangan tersebut, buat index.html, css/style.css, dan js/script.js yang lengkap dan dapat dijalankan dengan membuka index.html. Jangan gunakan placeholder atau pseudocode. Gunakan bahasa Indonesia untuk teks antarmuka.
Hasil yang diharapkan	Tiga berkas yang saling terhubung dan berjalan.
Cara evaluasi	Menjalankan di browser, memeriksa konsol, menelusuri jalur berkas, dan membaca kode untuk fitur yang hilang.
Keputusan mahasiswa	Menerima kerangka, tetapi menandai bagian yang perlu ditinjau (validasi dan pembuatan id).

5.3 Memperbaiki Tampilan Antarmuka

Aspek	Uraian
Tujuan instruksi	Memperbaiki Tampilan Antarmuka
Contoh prompt	Perbaiki CSS agar responsif: satu kolom pada layar ≤ 800 px dan dua kolom pada layar lebih lebar. Pastikan kontras warna memadai, fokus keyboard terlihat, dan tidak ada scroll horizontal pada lebar 375 px. Hanya ubah css/style.css.
Hasil yang diharapkan	CSS yang responsif dan lebih mudah diakses.
Cara evaluasi	Menguji dengan mode perangkat seluler DevTools dan navigasi Tab (skenario MT-10).
Keputusan mahasiswa	Menerima perubahan tata letak; menolak saran warna yang kontrasnya rendah.

5.4 Menganalisis Kesalahan Program

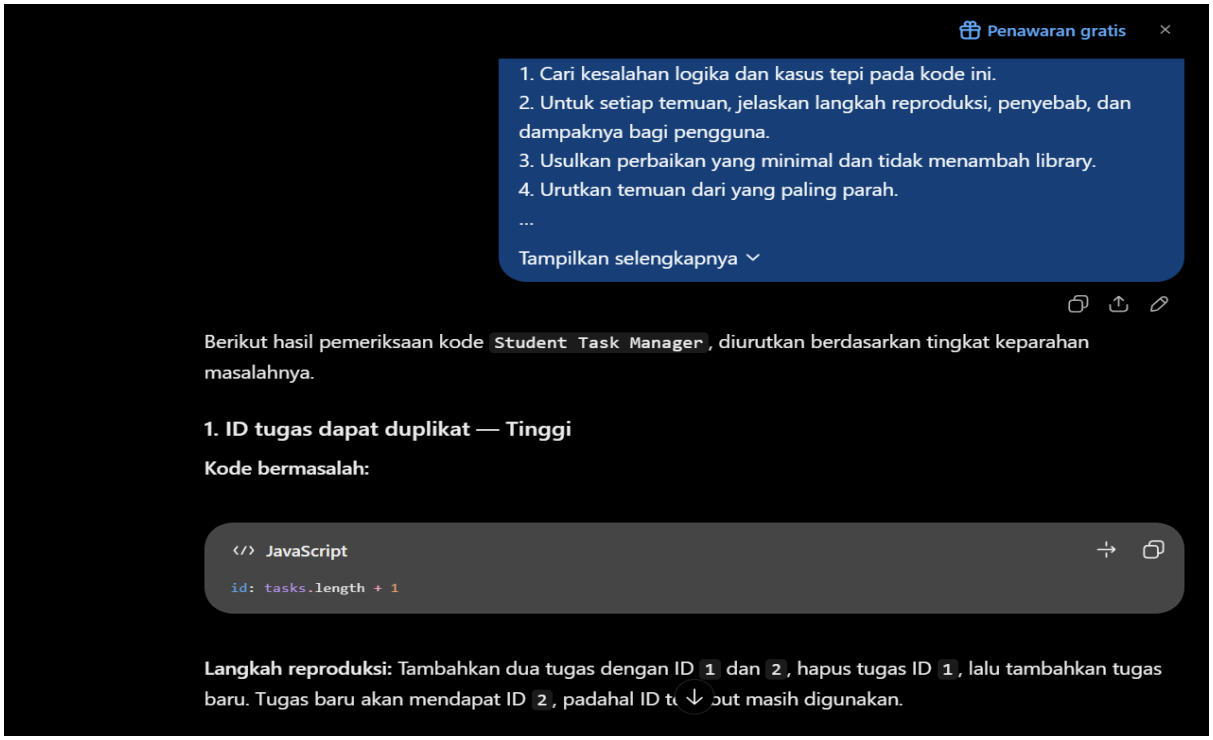
Aspek	Uraian
Tujuan instruksi	Menganalisis Kesalahan Program
Contoh prompt	Berikut draf fungsi addTask dan deleteTask (tempel kode). Cari kesalahan logika dan kasus tepi. Untuk tiap temuan, berikan langkah reproduksi, penyebab, dan usulan perbaikan minimal.
Hasil yang diharapkan	Daftar kesalahan dengan reproduksi dan penyebab.
Cara evaluasi	Mereproduksi sendiri setiap temuan dengan skrip (tests/demo-initial-bugs.js) sebelum memperbaiki.
Keputusan mahasiswa	Menerima temuan yang berhasil direproduksi; temuan yang tidak dapat direproduksi dicatat sebagai belum terbukti.

5.5 Memperbaiki Validasi dan Penyimpanan Data

Aspek	Uraian
Tujuan instruksi	Memperbaiki Validasi dan Penyimpanan Data
Contoh prompt	Perbaiki validasi agar menolak nama kosong atau hanya spasi, tanggal yang tidak ada di kalender, dan teks melebihi batas. Perbaiki penyimpanan agar aman terhadap JSON rusak dan kegagalan setItem. Tampilkan hasil sebagai fungsi murni yang bisa diuji tanpa DOM.
Hasil yang diharapkan	validateTask, loadTasks, dan saveTasks yang lebih tangguh.
Cara evaluasi	Menjalankan TC-02, TC-07 s.d. TC-11 dan menguji manual MT-02, MT-07, MT-09.
Keputusan mahasiswa	Menerima; menolak usulan library tanggal karena menambah dependensi yang tidak diperlukan.

5.6 Meninjau Ulang Kualitas Kode

Aspek	Uraian
Tujuan instruksi	Meninjau Ulang Kualitas Kode
Contoh prompt	Tinjau js/script.js dari sisi keterbacaan, keamanan DOM (XSS), konsistensi penamaan, dan penanganan kesalahan. Berikan temuan berurutan menurut tingkat keparahan; jangan menulis ulang seluruh berkas.
Hasil yang diharapkan	Daftar temuan terprioritas.
Cara evaluasi	Memeriksa tiap temuan pada kode; memastikan tidak ada penggunaan innerHTML untuk data pengguna.
Keputusan mahasiswa	Menerima penggunaan textContent; mengabaikan saran refaktor besar yang tidak diperlukan.



Gambar 4. Contoh Percakapan dengan AI saat Membuat atau Memperbaiki Kode

Keterangan: Bukti bahwa instruksi diberikan kepada AI dan keluarannya dievaluasi.

6. CONTOH PEMROGRAMAN

Potongan berikut diambil langsung dari js/script.js versi akhir sehingga konsisten dengan implementasi. Logika data sengaja dipisahkan dari kode antarmuka agar dapat diuji tanpa browser.

6.1 Validasi Input

```
function isValidDateString(value) {
  if (typeof value !== 'string' || !/^\d{4}-\d{2}-\d{2}$/.test(value)) { return false; }
  var p = value.split('-').map(Number);
  var d = new Date(p[0], p[1] - 1, p[2]);
  return d.getFullYear() === p[0] && d.getMonth() === p[1] - 1 && d.getDate() === p[2];
}

function text(value) {
  return value === undefined || value === null ? '' : String(value).trim();
}

function validateTask(input) {
  input = input || {};
  var values = { name: text(input.name), course: text(input.course), deadline: text(input.deadline) };
  var errors = {};

  if (!values.name) { errors.name = 'Nama tugas wajib diisi.'; }
  else if (values.name.length > LIMITS.name) { errors.name = 'Nama tugas maksimal ' + LIMITS.name + ' karakter.'; }

  if (!values.course) { errors.course = 'Mata kuliah wajib diisi.'; }
  else if (values.course.length > LIMITS.course) { errors.course = 'Mata kuliah maksimal ' + LIMITS.course + ' karakter.'; }

  if (!values.deadline) { errors.deadline = 'Tenggat waktu wajib diisi.'; }
  else if (!isValidDateString(values.deadline)) { errors.deadline = 'Tenggat waktu tidak valid.'; }

  return { valid: Object.keys(errors).length === 0, errors: errors, values: values };
}
```

Fungsi validateTask memangkas spasi (trim), memeriksa kolom wajib dan batas panjang, lalu mengembalikan objek berisi status valid, pesan kesalahan per kolom, dan nilai yang telah dibersihkan. isValidDateString tidak hanya mencocokkan pola YYYY-MM-DD, tetapi juga membuat objek Date dan membandingkan kembali komponen tanggalnya sehingga tanggal seperti 2026-02-30 ditolak. Pendekatan ini dipilih karena sederhana, tanpa library, dan mudah diuji.

6.2 Operasi Data yang Tidak Mengubah Array Asli

```
38  function generateId() {
39    return Date.now().toString(36) + '-' + Math.random().toString(36).slice(2, 8);
40  }
41
42  function addTask(tasks, values, id) {
43    var task = { id: id || generateId(), name: values.name, course: values.course, deadline: values.deadline, done: false };
44    return tasks.concat([task]);
45  }
46
47  function updateTask(tasks, id, values) {
48    return tasks.map(function (t) {
49      return t.id === id ? { id: t.id, name: values.name, course: values.course, deadline: values.deadline, done: t.done } : t;
50    });
51  }
52
53  function deleteTask(tasks, id) {
54    return tasks.filter(function (t) { return t.id !== id; });
55  }
```

Operasi data mengembalikan array baru (immutable) sehingga lebih mudah diuji dan terhindar dari efek samping. Id dibuat dari waktu dan angka acak, bukan dari jumlah item, untuk menghindari id ganda (lihat Bagian 8).

6.3 Penyimpanan dengan localStorage

```
95  function loadTasks(storage) {
96    try {
97      var raw = storage.getItem(STORAGE_KEY);
98      if (!raw) { return []; }
99      var data = JSON.parse(raw);
100     return Array.isArray(data) ? data.filter(isTaskShape) : [];
101   } catch (e) {
102     return [];
103   }
104 }
105
106 function saveTasks(storage, tasks) {
107   try {
108     storage.setItem(STORAGE_KEY, JSON.stringify(tasks));
109     return true;
110   } catch (e) {
111     return false;
112   }
113 }
```

loadTasks membungkus pembacaan dan parsing JSON dengan try/catch serta menyaring entri yang bentuknya tidak sesuai, sehingga data rusak tidak membuat aplikasi gagal. saveTasks mengembalikan false bila penyimpanan gagal (misalnya kuota penuh atau mode privat) agar antarmuka dapat menampilkan pesan.

6.4 Penampilan Data secara Aman

```
196     var name = document.createElement('div');
197     name.className = 'task-name';
198     name.textContent = task.name;
```

Teks dari pengguna ditampilkan dengan textContent sehingga karakter HTML diperlakukan sebagai teks biasa. Ini mencegah penyisipan elemen atau skrip melalui nama tugas.

7. EVALUASI DAN PENGUJIAN

Pengujian dibagi dua. Pertama, pengujian otomatis logika data yang benar-benar dijalankan dengan perintah node tests/test.js; hasilnya 12 dari 12 skenario lulus. Kedua, pengujian manual di browser yang belum dilakukan pada saat dokumen ini disusun; statusnya "Belum diverifikasi" dan harus diisi penulis setelah menjalankan skenario di browser. Rincian lengkap tersedia pada docs/test-plan.md.

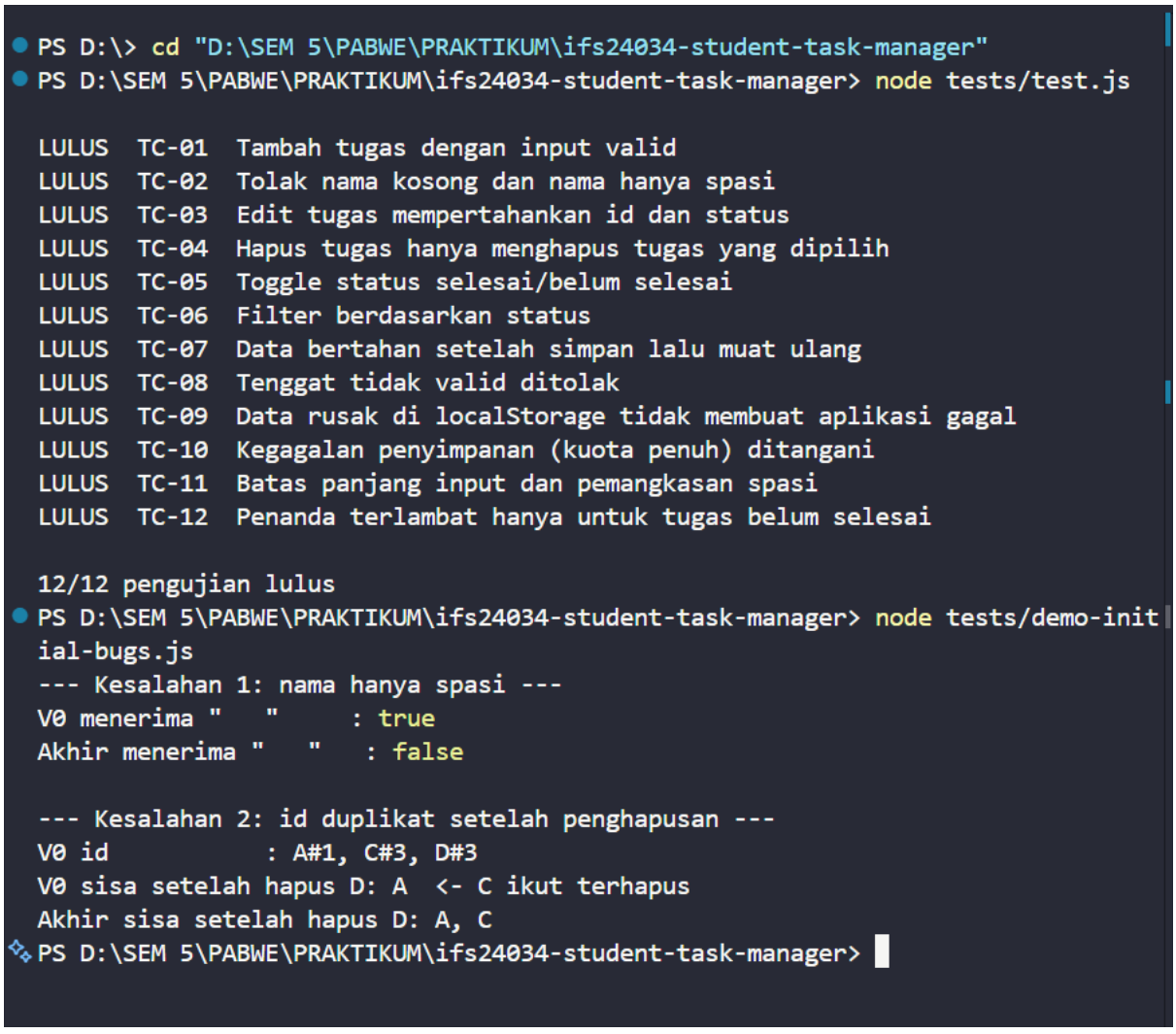
7.1 Pengujian Otomatis (sudah dijalankan)

ID	Skenario	Langkah	Hasil diharapkan	Hasil aktual	Status
TC-01	Tambah tugas valid	Jalankan node tests/test.js	Validasi lolos; tugas tersimpan, status belum selesai	LULUS	Lulus
TC-02	Nama kosong / hanya spasi	Jalankan node tests/test.js	Validasi gagal, pesan kesalahan nama	LULUS	Lulus
TC-03	Edit tugas	Jalankan node tests/test.js	Data berubah; id dan status tetap	LULUS	Lulus
TC-04	Hapus tugas	Jalankan node tests/test.js	Hanya tugas terpilih yang hilang	LULUS	Lulus
TC-05	Ubah status	Jalankan node tests/test.js	Status bergantian selesai/belum	LULUS	Lulus
TC-06	Filter status	Jalankan node tests/test.js	Semua/aktif/selesai sesuai	LULUS	Lulus
TC-07	Simpan dan muat ulang	Jalankan node tests/test.js	Data yang dimuat identik	LULUS	Lulus
TC-08	Tenggat tidak valid	Jalankan node tests/test.js	Format salah dan 2026-02-30 ditolak; 2028-02-29 diterima	LULUS	Lulus
TC-09	Data rusak di storage	Jalankan node tests/test.js	Daftar kosong / entri valid saja, tanpa error	LULUS	Lulus
TC-10	Penyimpanan gagal	Jalankan node tests/test.js	saveTasks mengembalikan false	LULUS	Lulus
TC-11	Batas panjang dan trim	Jalankan node tests/test.js	Nama > 100 dan mapel > 60 ditolak; spasi tepi dipangkas	LULUS	Lulus
TC-12	Penanda terlambat	Jalankan node tests/test.js	Hanya tugas belum selesai yang lewat tenggat	LULUS	Lulus

7.2 Pengujian Manual di Browser (belum dilakukan)

ID	Skenario	Langkah	Hasil diharapkan	Hasil aktual	Status
MT-01	Tambah tugas valid	Isi 3 kolom, klik Tambah Tugas	Tugas muncul; pesan sukses	[isi]	Belum diverifikasi
MT-02	Nama kosong	Kosongkan nama, klik Tambah	Pesan "Nama tugas wajib diisi."	[isi]	Belum diverifikasi
MT-03	Edit tugas	Klik Edit, ubah nama, simpan	Nama baru tampil	[isi]	Belum diverifikasi
MT-04	Hapus tugas	Klik Hapus, konfirmasi	Tugas hilang; batal = tetap	[isi]	Belum diverifikasi

ID	Skenario	Langkah	Hasil diharapkan	Hasil aktual	Status
MT-05	Ubah status	Centang kotak tugas	Nama dicoret; lencana Selesai	[isi]	Belum diverifikasi
MT-06	Filter status	Klik tombol filter	Daftar sesuai status	[isi]	Belum diverifikasi
MT-07	Persistensi	Tambah data, tekan F5	Data tetap ada	[isi]	Belum diverifikasi
MT-08	Tenggat kosong	Kosongkan tenggat, klik Tambah	Pesan tenggat wajib diisi	[isi]	Belum diverifikasi
MT-09	Data rusak	Set localStorage rusak, F5	Halaman tetap tampil, daftar kosong	[isi]	Belum diverifikasi
MT-10	Responsif	Lebar ±375 px	Satu kolom, tanpa scroll horizontal	[isi]	Belum diverifikasi



Gambar 5. Hasil Pengujian Aplikasi setelah Perbaikan

Sumber: [ISI: screenshot keluaran node tests/test.js dan pengujian manual, berkas docs/screenshots/05-pengujian-otomatis.png]

Keterangan: Bukti eksekusi pengujian setelah perbaikan diterapkan. Gambar ini masih placeholder; ambil screenshot sendiri dari terminal dan browser.

8. IDENTIFIKASI KESALAHAN DAN PERBAIKAN

Bagian ini membahas dua kesalahan pada draf awal (V0) yang direproduksi dengan skrip tests/demo-initial-bugs.js. Draf V0 disusun sebagai representasi draf pertama yang sederhana, dan kesalahannya dijalankan sebenarnya; penulis dapat menambahkan kesalahan yang muncul dari sesi AI pribadinya.

8.1 Kesalahan 1: Id Ganda setelah Penghapusan

Kode awal (sebelum):

```
function addTaskV0(tasks, name, course, deadline) {
  if (name == '') { return false; }
  tasks.push({ id: tasks.length + 1, name, course, deadline, done: false });
  return true;
}
function deleteTaskV0(tasks, id) { return tasks.filter((t) => t.id !== id); }
```

Cara mereproduksi: Tambahkan tugas A, B, C (id 1, 2, 3); hapus B; tambahkan D. Id D = jumlah item + 1 = 3, sama dengan id C.

Hasil eksekusi: Daftar id menjadi “A#1, C#3, D#3”. Ketika pengguna menghapus D, tugas C ikut terhapus; sisa daftar hanya “A”.

Penyebab: Id diturunkan dari panjang array yang berubah ketika ada penghapusan, sehingga tidak unik.

Perbaikan (sesudah): Id dibuat oleh generateId() dan operasi bersifat immutable.

```
38  function generateId() {
39    return Date.now().toString(36) + '-' + Math.random().toString(36).slice(2, 8);
40  }
41
42  function addTask(tasks, values, id) {
43    var task = { id: id || generateId(), name: values.name, course: values.course, deadline: values.deadline, done: false };
44    return tasks.concat([task]);
45  }
```

Hasil setelah perbaikan: Pada skenario yang sama, sisa daftar setelah menghapus D adalah “A, C”.

Alasan memilih solusi: Id berbasis waktu dan angka acak tidak bergantung pada jumlah item serta tidak memerlukan state tambahan. Alternatif penghitung yang disimpan ditolak karena menambah data yang harus disinkronkan. Untuk skala aplikasi ini peluang tabrakan id dapat diabaikan; pada sistem multi-pengguna diperlukan id dari server.

8.2 Kesalahan 2: Validasi Menerima Nama Berisi Spasi

Kode awal (sebelum): if (name == '') { return false; }

Hasil eksekusi: V0 menerima nama “ ” (tiga spasi): true. Versi akhir menolaknya: false.

Penyebab: Pengecekan hanya membandingkan dengan string kosong tanpa memangkas spasi.

Perbaikan (sesudah): Nilai dipangkas dengan trim() sebelum diperiksa, seperti pada validateTask (Bagian 6.1). Perilaku ini dicakup TC-02 dan TC-11.

Alasan memilih solusi: Perbaikan terpusat pada satu fungsi validasi yang mudah diuji dan sekaligus menyimpan nilai yang sudah dibersihkan.

8.3 Temuan yang Perlu Diverifikasi Manual

Penggunaan innerHTML untuk menampilkan teks pengguna berpotensi menimbulkan XSS. Temuan ini berasal dari analisis kode dan belum direproduksi di browser dalam penyusunan dokumen ini. Versi akhir memakai textContent; penulis perlu memverifikasinya dengan memasukkan uji sebagai nama tugas, yang harus tampil sebagai teks apa adanya.

9. KESIMPULAN

Proyek ini menargetkan aplikasi Student Task Manager yang memenuhi fitur minimal: tambah, tampil, edit, hapus, status selesai, validasi, filter, penyimpanan localStorage, dan tampilan responsif. Pengujian otomatis logika data lulus 12 dari 12 skenario, sedangkan pengujian antarmuka di browser masih harus diverifikasi secara manual.

AI generatif mempercepat perancangan, penyusunan kode awal, dan peninjauan, serta membantu menemukan kasus tepi. Meski demikian, AI memiliki keterbatasan: keluarannya dapat mengandung kesalahan logika (misalnya id ganda dan validasi yang lemah), tidak mengetahui konteks lingkungan eksekusi, dan dapat memberi saran yang berlebihan seperti penambahan library yang tidak diperlukan. Karena itu verifikasi manual, pembacaan kode, dan pengujian terukur tetap wajib, dan tanggung jawab atas hasil akhir berada pada pengembang.

10. DAFTAR PUSTAKA

[1] MDN Web Docs. “HTML: HyperText Markup Language.” Mozilla. https://developer.mozilla.org/en-US/docs/Web/HTML

[2] MDN Web Docs. “CSS: Cascading Style Sheets.” Mozilla. https://developer.mozilla.org/en-US/docs/Web/CSS

[3] MDN Web Docs. “JavaScript.” Mozilla. https://developer.mozilla.org/en-US/docs/Web/JavaScript

[4] MDN Web Docs. “Web Storage API.” Mozilla. https://developer.mozilla.org/en-US/docs/Web/API/Web_Storage_API

[5] Node.js. “Assert | Node.js Documentation.” OpenJS Foundation. https://nodejs.org/api/assert.html

[6] GitHub. https://github.com/Tomannn-star/ifs24034-student-task-manager